

MANUAL DEL PROGRAMADOR: Módulo de Conversión Hexadecimal

1. Contexto Arquitectónico y Control de Versiones

Este documento detalla el funcionamiento de la clase Calculadora_Hexadecimal. Según la topología del repositorio, este componente se desarrolla de forma aislada en la rama `conversion_hexadecimal`.

Dentro del diagrama de clases general (Calculadora de bases 2, 8, 10, 16), este módulo está diseñado para funcionar como una **Clase de Utilidad (Utility Class)** o un servicio algorítmico. Su objetivo es aislar la lógica matemática de la base 16 antes de su integración (merge) a la rama main y su posterior conexión con la rama interfaz-visual.

2. Descripción General del Módulo

La clase implementa algoritmos de conversión de bajo nivel entre bases numéricas (Decimal a Hexadecimal y viceversa). El módulo destaca por prescindir de las funciones de conversión nativas de Java (como `Integer.toHexString()`), implementando el control manual del flujo de datos, el cálculo de residuos y el análisis posicional de cadenas.

3. Diccionario de Métodos

3.1. `convertirDecimalAHexadecimal(int numeroDecimal)`

Transforma un número entero en base 10 a su representación alfanumérica en base 16.

- **Parámetros:** `numeroDecimal` (Tipo `int`) - El número a convertir.
- **Retorno:** `String` - La cadena resultante en base 16.
- **Lógica Algorítmica:**
 1. **Diccionario de Búsqueda:** Declara un arreglo de caracteres (`simbolosHex`) para el mapeo directo de los residuos lógicos (índices del 0 al 15) a sus correspondientes caracteres ('0'-'9', 'A'-'F').
 2. **Manejo de Casos Extremos:** Retorna "0" inmediatamente si la entrada es cero.
 3. **Extracción por División Sucesiva:** Implementa un bucle que calcula el módulo 16 (`% 16`) del número para obtener el dígito menos significativo, mapea el residuo, y lo concatena al inicio de la cadena de resultado. El número se divide entre 16 en cada iteración hasta llegar a cero.
- **Complejidad Computacional:** $O(\log_{16} N)$, donde N es el valor del entero ingresado.

3.2. `convertirHexadecimalADecimal(String numeroHexadecimal)`

Transforma una cadena que representa un valor hexadecimal a su equivalente entero en base 10.

- **Parámetros:** `numeroHexadecimal` (Tipo `String`) - La cadena a procesar.
- **Retorno:** `int` - El valor numérico base 10. Retorna -1 como código de error si la cadena contiene caracteres no válidos.

- **Lógica Algorítmica:**
 1. **Estandarización:** Convierte el String a mayúsculas para simplificar las operaciones aritméticas con la tabla ASCII.
 2. **Análisis Posicional Inverso:** Itera la cadena desde el índice `length() - 1` hasta 0.
 3. **Conversión ASCII a Valor:**
 - Dígitos 0-9: Se resta el valor ASCII de '0'.
 - Letras A-F: Se resta el valor ASCII de 'A' y se suma un offset de 10.
 4. **Acumulación:** Multiplica el valor del carácter por su valorPosicional (potencias de 16) y actualiza el acumulador general. El multiplicador posicional se incrementa por un factor de 16 en cada ciclo.
- **Complejidad Computacional:** $O(L)$, donde L es la longitud del String ingresado.

3.3. `main(String[] args)`

Actualmente, funciona como un entorno de prueba por consola (CLI) dentro de la rama.

- **Funcionalidad:** Instancia un Scanner e implementa un bucle do-while con una estructura switch para validar los métodos estáticos descritos anteriormente.

4. Guía de Integración (Merge a main)

Dado que el proyecto contará con una interfaz-visual y englobará múltiples bases, se deben contemplar los siguientes pasos al momento de fusionar esta rama:

1. **Eliminación del Entorno de Prueba:** El método `main` y las importaciones de `java.util.Scanner` deberán ser removidos o movidos a una clase de pruebas unitarias (`Calculadora_HexadecimalTest`), ya que el ciclo de vida del programa será controlado por la interfaz gráfica o un controlador principal.
2. **Manejo de Errores Orientado a Objetos:** En lugar de imprimir "Numero hexadecimal invalido." en consola y retornar -1 (que podría confundirse con el valor hexadecimal -1 si el sistema admitiera negativos), se recomienda refactorizar el método `convertirHexadecimalADecimal` para que lance una excepción formal (por ejemplo, `IllegalArgumentException("Caracter no válido en cadena hexadecimal")`). Esto permitirá que la rama interfaz-visual capture el error mediante un bloque try-catch y muestre un cuadro de diálogo al usuario.
3. **Estandarización de Interfaz (Opcional):** Si la arquitectura general lo requiere, esta clase podría implementar una interfaz genérica (ej. `IConversorBase`) para estandarizar las firmas de los métodos junto con las clases de las ramas `conversion_binaria` y `conversion_octal`.